Walks that Forget the Cycles

A machine-checked bijection between tree-like walks of a graph and walks on Godsil's path tree, in Lean 4

Carles Marín*

Independent researcher

karlesmarin@gmail.com

June 2026

Abstract

This is the third piece of a series that builds, in Lean 4 / Mathlib, the two sides of the matching/Ihara trace formula. In *Unfolding a Graph into a Tree* [13] I used Godsil's *path tree* $T(G, v)$ to prove that the matching polynomial of a graph is real-rooted: the path tree is a forest, and a forest carries a symmetric spectrum. Here I put the *same forest* to a second use. A closed walk of G is *tree-like* when it never knowingly closes a cycle—at every step it either advances to a fresh vertex or backtracks to its parent, exactly as if it were walking on a tree. I give a machine-checked, `sorry`-free proof that the tree-like closed walks of G based at v are in length-preserving bijection with the closed walks of $T(G, v)$ based at its root, realized by the down-projection $\pi\colon T(G, v) \to G$ that sends a path to its endpoint. As a corollary the tree-like walk count becomes a sum of adjacency-matrix powers of the path trees,

$$\mathrm{treeLikeWalkCount}(G, k) \;=\; \sum_{v} \big[A(T(G, v))^k\big]_{\mathrm{root},\mathrm{root}},$$

and, since each $T(G, v)$ is a forest, its matching polynomial equals the characteristic polynomial of its adjacency matrix. I have not found a prior formalization of Godsil's path tree as a walk-counting object, of the resulting bijection, or of the intrinsic "tree-like" predicate, in any proof assistant. I am exact about the boundary: this paper builds the *combinatorial* half of Godsil's moment theorem $p_k = \sum_i \theta_i^k = \mathrm{treeLikeWalkCount}(G, k)$. The remaining *spectral* half—the single-entry resolvent identity and the univariate Newton step that turn $[A(T)^k]_{\mathrm{root}}$ into a coefficient of $\mu(G - v)/\mu(G)$—is mapped in Section 7, not built. None of this is new mathematics; the contribution is the formalization, and the fact that the path tree now counts walks in the same library where it already diagonalizes a spectrum.

Reader's map. If you read only one thing, read Theorem 1—the bijection—and the one-line reason it holds: the path tree is the graph *unrolled* until paths self-intersect, an *immersion* of G in which a walk lifts at most one way, and lifts at all exactly when it is tree-like. Everything else is the careful version of that sentence: a local-injectivity lemma (Section 4), a `liftSeq`↔path invariant that makes "tree-like" computable and matches it to the lift (Section 5), and the lift itself (Section 6). The honest limit—the spectral half is mapped, not built—is stated as plainly as the result.

*Formalized with AI assistance (Claude, Anthropic); the mathematics and all claims are the author's responsibility. The methodology is discussed in Section 8.

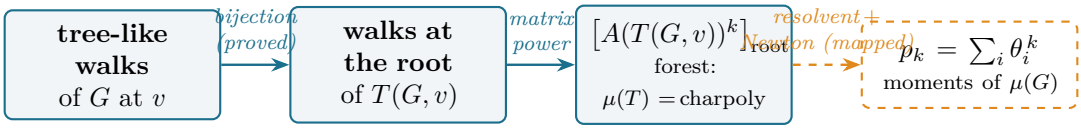

Figure 1: The route of the paper, and where it stops. The tree-like walk count of G equals the walk count at the root of Godsil's path tree (the bijection, Theorem 1), which is a power of the tree's adjacency matrix (Theorem 2); and each path tree is a forest, so that matrix's characteristic polynomial is the matching polynomial (Theorem 3). The solid arrows are machine-checked here; the dashed arrow to the moments p_k—the single-entry resolvent, Godsil's ratio identity, and the univariate Newton step—is mapped in Section 7, not built.

1 Introduction: one tree, two harvests

When does a walk on a graph fail to notice that the graph has cycles—and why should counting those forgetful walks recover the moments of a polynomial that is the characteristic polynomial of no matrix at all? The two halves of that question turn out to have a single answer, a tree; this paper machine-checks the combinatorial half of it.

Answering it—and explaining why a single combinatorial device settles two facts about a graph that look unrelated—asks three threads of history to meet.

A polynomial with real roots and no matrix (1972). A finite graph G has a matching polynomial $\mu(G,x) = \sum_k (-1)^k m_k(G)\, x^{n-2k}$, where $m_k(G)$ counts its k-edge matchings [2, 3]. In 1972 Heilmann and Lieb, studying monomer–dimer systems in statistical mechanics, proved that its roots are always real [1]—a phase-transition statement in physics, a spectral one in combinatorics. But real roots are the signature of a *symmetric matrix*, and $\mu(G)$ is the characteristic polynomial of no fixed matrix unless G is a forest. So the question sharpens: if there is no matrix, where do the real roots, and the combinatorial content of their power sums $p_k = \sum_i \theta_i^k$, come from?

One tree, two theorems (1981). Godsil's answer was a single device [4, 5, 6]. Given a vertex v, the *path tree* $T(G,v)$ has as vertices the simple paths of G that start at v, with one path joined to another when the second extends the first by a single edge. It is a forest—the finite truncation of the *universal cover* of G unrolled from v—and from it Godsil drew two conclusions. First, $\mu(G) \mid \mu(T(G,v))$, and a forest's matching polynomial *is* the characteristic polynomial of its adjacency matrix, hence real-rooted; this recovers Heilmann–Lieb, and is what Part II of this series [13] formalized. Second—the subject of this paper—the path tree *counts walks*.

Walks, covers, and a trace formula. The universal cover is the deeper object behind the finite tree: the spectral radius of the infinite $(\Delta-1)$-regular tree is $2\sqrt{\Delta-1}$, the *Ramanujan* edge, which is also Godsil's bound on the matching roots, and the cover's spectrum is governed by the matching polynomial [7, 8]. On the finite side this is a *trace formula*: $\operatorname{tr}(A^k)$ counts *all* closed walks of G, while p_k counts only the *tree-like* ones; below the girth every walk is tree-like, so the two agree, and the first gap counts shortest cycles. The cycle side, $\operatorname{tr}(A^k)$ and its non-backtracking refinement, is the *Bass* companion of this series [14, 9, 10]; this paper builds the combinatorial half of the tree side.

A closed walk of G is *tree-like* when, viewed step by step, it only ever extends to a new vertex or retreats to the vertex it came from: it never traverses an edge back to an earlier, non-parent vertex, so it never knowingly encloses a cycle. (It may still, in aggregate, traverse the three edges

of a triangle—as long as each intermediate *path* stays simple; this is exactly where the naive definition fails, Figure 2.) Godsil's observation is that these are precisely the walks G inherits from $T(G, v)$: a closed walk at the root of the path tree, pushed down to G by recording only the current endpoint, is a tree-like closed walk at v, and every tree-like walk arises this way, uniquely (Figure 3). Summing over v and over the length-k walks gives treeLikeWalkCount(G, k), and the moment theorem identifies it with $p_k(G)$.

Why formalize it. The matching polynomial sits at the intersection of three of this series' threads—real-rootedness (Parts I–II [12, 13]), the matching/Ihara trace formula (the *Bass* companion [14]), and, through the band edge $2\sqrt{\Delta - 1}$, Ramanujan graphs [11]. The trace formula has a *cycle* side, $\mathrm{tr}(A^k)$ counting all closed walks, and a *tree* side, p_k counting tree-like ones; below the girth they coincide, and the first gap is a count of shortest cycles. To state that bridge inside a proof assistant one needs the tree side as an honest object, and the tree side is precisely a statement about the path tree. Part II built the path tree for its spectrum; this paper teaches the same Lean object to count.

What is new here, and what is not. The mathematics is Godsil's. The contribution is the machine-checked construction: the path tree as a walk-counting graph, the down-projection as a graph homomorphism that is an *immersion*, an intrinsic decidable predicate `IsTreeLike` computing the lift as a stack discipline, the invariant tying that stack to the current path-tree vertex, the lift in the surjective direction, and the resulting cardinality bijection. For none of these could I find a prior formalization in any proof assistant. The headline depends only on the three standard axioms (`propext`, `Classical.choice`, `Quot.sound`); there is no `sorry`.

2 The two sides, kept apart

What, exactly, are we matching—and how do we keep the two sides from being the same object wearing two hats? A bijection only says something when its ends are defined without reference to each other; so I fix both, and only later let them meet. Throughout, G is a `SimpleGraph` on a vertex type V, and $v \in V$ is a base point. I reuse the path tree of Part II verbatim.

Definition 1 (Path tree, `pathTree`)**.** *The vertices of $T(G, v) =$ `pathTree` $G\, v$ are the simple paths of G starting at v (`PathFrom` $v = \Sigma_u\, G.\mathrm{Path}\, v\, u$). Two are adjacent when one is obtained from the other by appending a single edge at its end (`Grows`). The root is the trivial path at v (`pathTreeRoot`). It is a forest: `pathTree_isAcyclic`.*

Definition 2 (Down-projection, `pathTreeProj`)**.** *$\pi\colon T(G, v) \to G$ sends a path to its endpoint, $\pi\langle u, p\rangle = u$. Adjacency in the path tree means one path extends the other by an edge, so the two endpoints are adjacent in G; hence π is a graph homomorphism, and $\pi(\mathrm{root}) = v$.*

The tree-like predicate is defined intrinsically, without reference to the path tree, so that the two sides of the bijection are genuinely different objects to be matched. A closed walk w at v is fed, vertex by vertex (its support past the first entry), to a deterministic *lift* `liftSeq` that threads a stack—the current simple path from v. For each next vertex c: if c is not on the stack, *push* it (an EXTEND); if c is the penultimate vertex of the stack, *pop* (a RETREAT to the parent); otherwise the lift *fails*. The two cases are mutually exclusive, so the lift is a fold.

Definition 3 (Tree-like, `Walk.IsTreeLike`)**.** *w is* tree-like *iff its lift, started from the root stack $[v]$, returns to $[v]$: `liftSeq` w.support.tail $[v] =$ `some` $[v]$. This is decidable.*

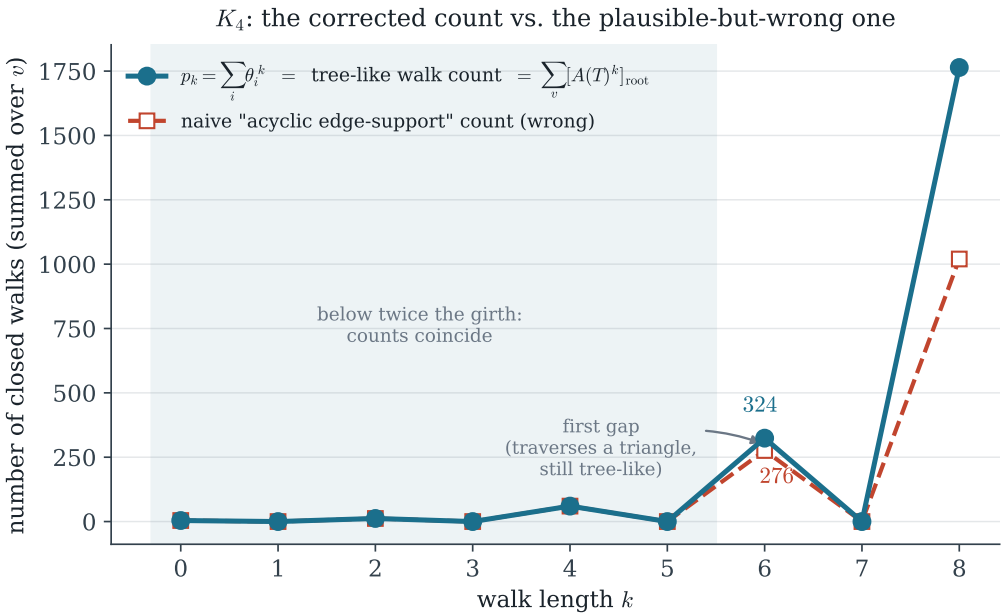

Figure 2: Why the definition matters, on K_4. The matching power sums p_k (teal) coincide with the tree-like / path-tree count for every k—this paper's bijection makes the second equality exact. The plausible "acyclic edge-support" count (red) agrees through $k = 5$ but then undercounts, 276 against 324 at $k = 6$, because a tree-like walk may traverse all three edges of a triangle while every intermediate path stays simple. A cheap numerical check caught the wrong definition before any formal effort was spent on it.

This definition is the corrected one. The naive choice—"the edge-support of w is acyclic"—is *wrong* for the moment theorem: on K_4 with $k = 6$ it undercounts (276 against $p_6 = 324$), because a tree-like walk may traverse a cycle's edges in aggregate. The `liftSeq` predicate was validated numerically against the matching power sums on P_4, C_4, C_5, K_4 and the Petersen and $K_{3,3}$ graphs for $k \leq 8$ before it was formalized (Figure 2).

3 The result

Theorem 1 (The bijection, `card_treeLike_eq_pathTreeWalks`)**.** *For every finite graph G, base point v, and length k, the map $W \mapsto W.\mathrm{map}\,\pi$ is a bijection from the closed walks of length k at the root of $T(G, v)$ onto the tree-like closed walks of length k of G at v. In particular their cardinalities agree:*

$$\#\{\,\textit{tree-like walks of } G \textit{ at } v,\ \textit{length } k\,\} \;=\; \#\{\,\textit{walks at the root of } T(G,v),\ \textit{length } k\,\}.$$

What does it take to trust a bijection? A map, and its promise kept in two directions. The map is π read on whole walks, $W \mapsto W.\mathrm{map}\,\pi$; its promise is three facts, and the three proof sections that follow are, in order, exactly those three. The map must *land where it should*: a projected tree-walk is always tree-like, so π carries the tree's walks *into* G's tree-like ones (Lemma 1, earned in Section 5). It must *forget nothing*: no two tree-walks cast the same shadow (Lemma 2, Section 4). And it must *miss nothing*: every tree-like walk is some walk's shadow—the lift exists (Lemma 3,

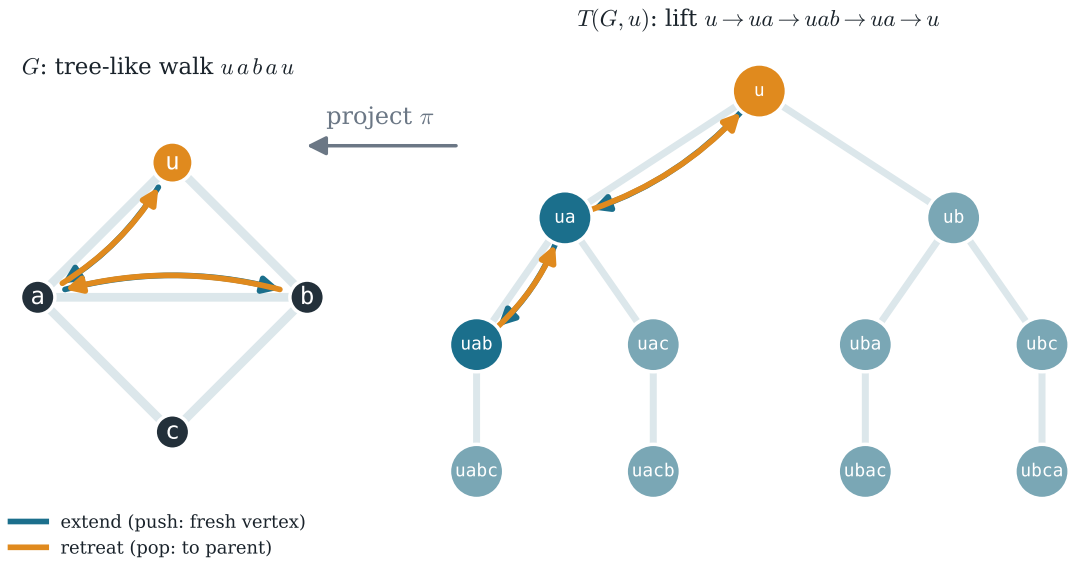

Figure 3: The bijection on the diamond $G = K_4 - e$, root u. A closed walk at the root of the path tree $T(G,u)$ (right), $u \to ua \to uab \to ua \to u$, projects under π (a path $\mapsto$ its endpoint) to a tree-like closed walk of G (left), $u\,a\,b\,a\,u$. Each step is an EXTEND (teal, advance to a fresh vertex / push) or a RETREAT (amber, backtrack to the parent / pop). The walk on the left traverses the edges ua, ab outward and retraces them back; the lift on the right makes the path history explicit, and is the unique walk over it.

Section 6). Land, injective, surjective; that is the whole of Theorem 1, and the rest of the paper is those three words made precise.

Lemma 1 (Image is tree-like, `pathTreeProj_map_isTreeLike`)**.** *The π-projection of any closed walk at the root of $T(G,v)$ is tree-like.*

Lemma 2 (Injectivity, `pathTreeProj_walk_injective`)**.** *$W \mapsto W.\mathrm{map}\,\pi$ is injective on walks of $T(G,v)$.*

Lemma 3 (Surjectivity / the lift, `exists_root_lift`)**.** *Every tree-like closed walk of G at v is $W.\mathrm{map}\,\pi$ for some closed walk W at the root of $T(G,v)$.*

A bijection that only renames a finite set is bookkeeping—so what does this one buy? Its worth is that the two sides are different *kinds* of object: on the left, a combinatorial count; on the right, walks on a *tree*. And a tree has a matrix. So the count on the right is a matrix power, a matrix power is a spectral quantity, and the rename has quietly moved us onto the spectral side, where the rest of the moment theorem lives. Two corollaries take the step.

Theorem 2 (Spectral form, `treeLikeWalkCount_eq_sum_pathTree_adjMatrix_pow`)**.**

$$\mathrm{treeLikeWalkCount}(G,k) \;=\; \sum_{v \in V} \left[A(T(G,v))^k\right]_{\mathrm{root,root}}.$$

Theorem 3 (Forest bridge, `matchingPoly_pathTree_eq_charpoly`)**.** *Each path tree is a forest, so its matching polynomial is the characteristic polynomial of its adjacency matrix: $\mu(T(G,v)) = \mathrm{charpoly}(A(T(G,v)))$.*

4 The path tree is an immersion

If the path tree is the graph unrolled, what stops a walk from unrolling ambiguously—from having two different lifts? The answer is local, and it is the whole engine of injectivity. The neighbours of a path-tree vertex p are its unique parent (drop the last edge) and its children (append one edge each). The projection π sends the parent to p's penultimate vertex, which lies on p, and each child to its fresh endpoint, which does not; and distinct children have distinct endpoints. So π *separates* the neighbours of every vertex: `pathTreeProj_injOn_neighborSet`. The parent half is Part II's `Grows.parent_unique`; its dual `Grows.child_unique` (a one-edge extension is determined by its endpoint) is new here.

This local injectivity is exactly the immersion property: π is locally injective but not locally surjective—a G-neighbour of p's endpoint that already lies on p has *no* preimage above p. That asymmetry is the whole story. It is also an old one in a different dress: a covering map, in the sense of Poincaré's topology, lifts *every* path uniquely; an immersion lifts only the paths that never double back into themselves—and "never doubles back into a non-parent vertex" is precisely what tree-like means. A walk in $T(G, v)$ is determined, step by step, by its π-image together with its start, because at each vertex the next path-tree vertex is forced by the local injection; an induction mirroring Mathlib's `Walk.map_injective_of_injective`, with the global hypothesis replaced by the local one, gives Lemma 2.

5 The `liftSeq`↔path invariant

`IsTreeLike` is a stack discipline on G alone—the path tree appears nowhere in it—so why should it detect exactly the walks that come from the tree? Because the stack *is* the tree vertex in disguise. Reading a walk as a sequence of pushes and pops is the oldest move in enumerative combinatorics—the ballot problem, von Dyck's balanced words, the Catalan numbers all live on the same stack—and here it is the bridge, in the form of a single invariant: as one walks $T(G, v)$, running `liftSeq` on the projected walk tracks the current path-tree vertex.

Lemma 4 (One step, `liftSeq_step`)**.** *For a path-tree edge $s \sim y$ and any tail M, `liftSeq` $(y_1 :: M)\, s$.support = `liftSeq` $M\, y$.support: a child step is a* PUSH*, a parent step is a* POP*.*

Lemma 5 (Invariant, `liftSeq_map_invariant`)**.** *For a path-tree walk $W : s \to x$, `liftSeq` $(W.\mathrm{map}\,\pi)$.support.tail* `some` x.support*.*

Lemma 1 is the case $s = x =$ root, whose path-support is $[v]$. The same invariant drives the lift: it is the bookkeeping that, run in reverse, reconstructs the path-tree walk from a tree-like walk.

6 The lift: climbing back into the tree

Every tree-like walk should come from the tree—but how do we exhibit the walk it comes from, when its very endpoints in the tree are not given but computed? Surjectivity is the only direction that builds a walk in the dependent type $(T(G, v))$.Walk, and its endpoints are computed, not given. The construction (`exists_lift`) is a recursion on the G-walk: the hypothesis "`liftSeq` accepts w from the current vertex" supplies, at each step, the EXTEND/RETREAT decision, so there is no failure branch. The current path-tree vertex is carried as the explicit triple $\langle a, pp, hpp\rangle$ (endpoint, path, proof-it-is-a-path), which sidesteps the heterogeneous equalities that a Σ-typed accumulator would force. Two further frictions are handled by the same trick: the recursion proves equality of

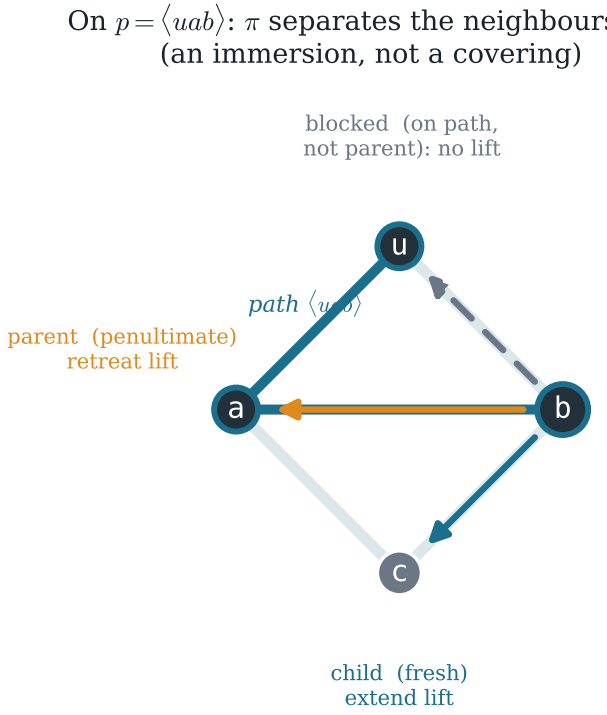

Figure 4: The immersion, at the path-tree vertex $p = \langle uab \rangle$ (the path $u\,a\,b$, teal). The next step looks at the neighbours of the endpoint b, which in G are a, c, u. Each falls into exactly one case: a is the penultimate of p (the *parent*, a retreat), c is fresh (a *child*, an extend), and u lies on p but is not its parent—*blocked*, with no lift above p. So π is injective on the neighbours of p but not surjective; the blocked direction is why the path tree is an immersion, and why a projected walk can only be tree-like.

supports (a `List` V, free of dependent types) rather than of walks, and `Walk.support_injective` upgrades support equality to walk equality once the endpoints are pinned down in the closed-walk corollary (Lemma 3). The lift returns to the root because its final path-support is $[v]$, forcing the final tree-vertex to be the trivial path.

The bijection (Theorem 1) is then `Finset.card_bij` on $W \mapsto W.\mathrm{map}\,\pi$, with Lemma 1 for well-definedness, Lemma 2 for injectivity, and Lemma 3 for surjectivity; lengths are preserved because map preserves them.

7 The boundary: what is built, and what is mapped

The bijection counts; the moment theorem equates that count with a spectrum. Where, precisely, does the machine-checked ground end and the mapped-but-unbuilt algebra begin? Godsil's moment theorem is

$$p_k(G) \;=\; \sum_i \theta_i^k \;=\; \mathrm{treeLikeWalkCount}(G, k).$$

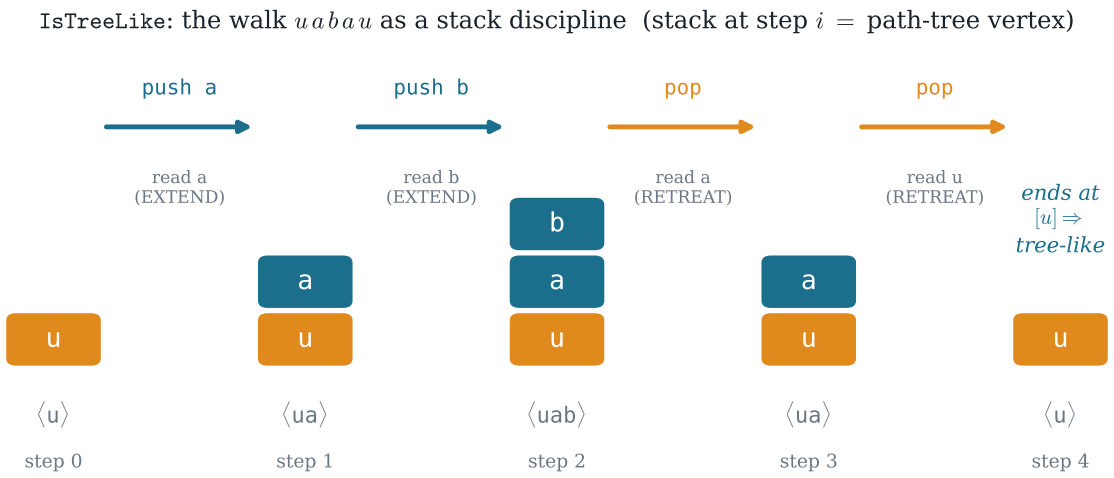

Figure 5: `IsTreeLike` as a stack discipline—the `liftSeq` predicate run on the walk $u\,a\,b\,a\,u$. The stack is the current simple path from u; each next vertex is read and either PUSHED (extend, teal: a fresh vertex) or POPPED (retreat, amber: it is the penultimate). The walk is tree-like iff the stack ends back at $[u]$. The stack at step i is exactly the path-tree vertex the lift sits on—$\langle u\rangle, \langle ua\rangle, \langle uab\rangle, \langle ua\rangle, \langle u\rangle$—which is the content of the invariant, Lemma 5.

This paper builds its *combinatorial* half—everything up to and including the spectral form (Theorem 2) and the forest bridge (Theorem 3). The *spectral* half is mapped here but not formalized; I state it plainly so the reader knows exactly where the machine-checked ground ends.

The remaining chain is standard algebra. For each path tree $T = T(G, v)$, the root–root resolvent generates the walk counts,

$$\sum_{k\geq 0} \left[A(T)^k\right]_{\mathrm{root}} X^k \;=\; \frac{\mathrm{charpolyRev}\big(A(T)_{\widehat{\mathrm{root}}}\big)}{\mathrm{charpolyRev}\big(A(T)\big)},$$

the single-entry case of $\mathrm{adj}(1 - XA) = \det(1 - XA)\,(1 - XA)^{-1}$, which this library already has in matrix form (`smul_resolventSeries_eq_adjugate`, `coeff_resolventSeries`). What is *not* yet in Mathlib, and is the one genuinely new linear-algebra lemma the completion needs, is the general principal-minor identity $\mathrm{adj}(M)_{ii} = \det\big(M\restriction_{j\neq i}\big)$ over an arbitrary finite index type (Mathlib has only the `Fin` cofactor expansion). Granting it, the forest bridge turns each charpolyRev into a reversed matching polynomial; Part II's `godsil_identity`, $\mu(G)\,\mu(T - \mathrm{root}) = \mu(G - v)\,\mu(T)$, rewrites the ratio as $\mu(G - v)/\mu(G)$; the vertex-deletion derivative law $\sum_v \mu(G - v) = X\,\mu'(G)$ (already proved, `sum_matchingPoly_deleteIncidenceSet`) sums it to $\mu'(G)/\mu(G)$; and the univariate Newton identity equates that logarithmic derivative with $\sum_k p_k X^k$. Matching coefficients of X^k closes the theorem. The last step is itself venerable: Newton's identities, relating a polynomial's power sums to its coefficients, were written down by Girard in 1629 and by Newton a generation later, and Sachs' 1964 coefficient theorem reads those coefficients off the graph's own subgraphs. Two of the links—the principal-minor lemma and the univariate-root Newton bridge—are general results absent from Mathlib, and both are kept in the repository's `MathlibPR` staging in canonical, graph-free form.

8 Implications and methodology

The author set the targets and adjudicated the mathematics; the assistant (Claude, Anthropic) drafted the proofs and this prose. Every theorem cited here compiles `sorry`-free against the pinned Mathlib revision [15] and depends only on the three standard axioms, which the reader can re-verify with `#print axioms`. The corrected "tree-like" definition is itself a small methodological datapoint: the natural acyclic-support reading is plausible, compiles, and is *false* for the theorem it is meant to serve; a cheap numerical check against the matching power sums caught it before any formal effort was spent, and the same check certifies the `liftSeq` definition that replaced it.

What it is good for—and what it is not. The objects here are not exotic. The matching polynomial is the monomer–dimer partition function of statistical mechanics [1]; the moments of its roots are the raw material of the method of moments for eigenvalue bounds; the bound those moments obey, $2\sqrt{\Delta - 1}$, is the threshold a graph must clear to be an expander; and the spectral object next door, the non-backtracking operator, sets a network's percolation threshold [9, 14]. This paper adds nothing to any of that. It only places one half of the bridge those methods cross—the tree side of the trace formula—on certified ground, in the same library as the cycle side, and leaves behind a small reusable object, the path tree, that already did one job (a real spectrum, Part II) and now does a second. No algorithm and no network: a foundation, and a modest one.

The mathematics is Godsil's, four decades old. I have only carried it, carefully, into a form a machine will check: in Part II the path tree carries a real spectrum, and here it counts the walks a graph mistakes for a tree. The cycle side of the matching/Ihara trace formula—$\mathrm{tr}(A^k)$—stands in the same library (the *Bass* companion [14]); should the spectral half of Section 7 one day be built, the tree side p_k would meet it, and the trace formula would be a single machine-checked line. That is left, without apology, for another day.

9 Questions left open

A formalization is also a list of the next questions, sharpened to the point where a machine could be asked them. These are the ones this paper leaves on the table.

- The completion of Section 7 hangs on one general, Mathlib-absent lemma: the principal-minor identity $\mathrm{adj}(M)_{ii} = \det(M \restriction_{j \neq i})$ over an arbitrary finite index type. Is the clean route a reindexing to `Fin` and the existing cofactor expansion, or is there a basis-free proof that never names an ordering?

- The corrected "tree-like" predicate is a *stack discipline*. Is it the unique reasonable intrinsic definition, or does the free-reduction-to-trivial reading (a walk that reduces to the empty word under backtrack cancellation) define the same class for every k—and if so, is the equivalence cheaper to prove than either side of the bijection?

- The bijection is length-preserving and vertex-local. Does it refine to an isomorphism of the *generating functions* before any spectral input, i.e. can $\sum_k (\#\text{tree-like}) X^k$ be identified with a path-tree resolvent entry purely combinatorially, leaving the matching polynomial out until the very end?

- Below the girth, tree-like walks are all walks, so the count equals $\mathrm{tr}(A^k)$ locally; the first gap, at $k = \text{girth}$, counts shortest cycles. Can the path-tree bijection be made to *see* that gap—to produce the cycle-counting correction term as the obstruction to a walk lifting?

- The same forest carries the band edge $2\sqrt{\Delta-1}$ (Part II) and, here, the walk counts whose growth rate is that very edge. Is the Ramanujan bound on the matching roots a *corollary* of this bijection plus a comparison with the infinite $(\Delta-1)$-ary tree, formalizable without re-entering the interlacing-families machinery?

And one to keep. *When a machine certifies that a graph's tree-like walks are exactly a tree's walks, has it proved that the graph is, for the purpose of counting, a tree—or only that we have named the precise sense in which it refuses to be one?*

References

[1] O. J. Heilmann and E. H. Lieb, *Theory of monomer–dimer systems*, Comm. Math. Phys. **25** (1972), 190–232.

[2] E. J. Farrell, *An introduction to matching polynomials*, J. Combin. Theory Ser. B **27** (1979), 75–86.

[3] L. Lovász and M. D. Plummer, *Matching Theory*, North-Holland Math. Stud. **121**, Amsterdam, 1986.

[4] C. D. Godsil, *Matchings and walks in graphs*, J. Graph Theory **5** (1981), 285–297.

[5] C. D. Godsil and I. Gutman, *On the theory of the matching polynomial*, J. Graph Theory **5** (1981), 137–144.

[6] C. D. Godsil, *Algebraic Combinatorics*, Chapman & Hall, New York, 1993.

[7] O. Angel, J. Friedman, and S. Hoory, *The non-backtracking spectrum of the universal cover of a graph*, Trans. Amer. Math. Soc. **367** (2015), 4287–4318.

[8] T. J. Spier, *Eigenvalues of universal covers and the matching polynomial*, preprint, 2025. `arXiv:2510.05041`.

[9] K. Hashimoto, *Zeta functions of finite graphs and representations of p-adic groups*, in *Automorphic Forms and Geometry of Arithmetic Varieties*, Adv. Stud. Pure Math. **15**, 1989, 211–280.

[10] H. Bass, *The Ihara–Selberg zeta function of a tree lattice*, Internat. J. Math. **3** (1992), 717–797.

[11] A. Marcus, D. A. Spielman, and N. Srivastava, *Interlacing families I: Bipartite Ramanujan graphs of all degrees*, Ann. of Math. **182** (2015), 307–325.

[12] C. Marín, *Random Signs into Matchings: the Godsil–Gutman estimator and real-rootedness in Lean 4* (Part I), 2026. Zenodo, DOI `10.5281/zenodo.20517350`.

[13] C. Marín, *Unfolding a Graph into a Tree: A Machine-Checked Proof of the Heilmann–Lieb Theorem in Lean 4* (Part II), 2026. Zenodo, DOI `10.5281/zenodo.20561832`.

[14] C. Marín, *Folding Edges into Vertices: A Machine-Checked Proof of Bass's Determinant Formula for the Ihara Zeta in Lean 4*, 2026. Zenodo, DOI `10.5281/zenodo.20573120`.

[15] The mathlib Community, *The Lean mathematical library*, in *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs* (CPP 2020), 367–381.